OpenLPT

Open-source 3D Lagrangian Particle Tracking

Shake-The-Box implementation with a Python GUI on a C++ kernel

Ni Research Lab · GWU

What is OpenLPT?

A research-grade pipeline for measuring **dense 3D particle motion** from a multi-camera setup.

- **Algorithm** — implementation of *Shake-The-Box* (Schanz et al., 2016) for high-density 3D particle tracking velocimetry
- **Architecture** — Python GUI (PySide6) drives a C++ kernel exposed through `pyopenlpt` bindings
- **Inputs** — synchronized camera frames (`.tif` / `.cine`) plus a calibration model (`lpt.Camera`)
- **Outputs** — long Lagrangian particle trajectories with per-frame `(x, y, z)` and optional bubble radius

The GUI pipeline

Five views in the left navigation, executed in order:

#	Stage	Module / View	What it produces
1	Calibration	`modules/camera_calibration`	`cam{i}.txt` per camera
2	Preprocessing	`modules/image_preprocessing`	processed frames + lists
3	Settings	`gui/views/tracking_settings_view.py`	`config.txt` + STB config
4	Tracking	`gui/views/tracking_view.py`	`Results/ConvergeTrack/*.csv`
5	Results	`modules/post_processing`	smoothed v / a, `.mat` / `.csv`

A side branch — **Volume Self-Calibration** (`modules/vsc`) — refines cameras using the

The Shake-The-Box algorithm

Initialization (frames 1 - 4) seeds short tracks via 2D peak detection, stereo triangulation, IPR, Shake refinement, and 4-frame predictive linking.

Convergence (frame n ≥ 5) runs a tight per-frame loop:

1. **Predict** — extrapolate `X(n)` for each active track (Wiener / polynomial)
2. **Project** — render each predicted particle through the **OTF** model
3. **Residual** — `R(u, v) = I(u, v) - sum_p OTF(p; X_p)`
4. **Shake** — perturb `X` to minimise local residual
5. **IPR on residual** — find new particles entering the volume
6. **Classify** track as *Active long*, *Inactive long*, or *Exit*

Getting started

```
# 1. Set up the conda environment
conda create -n OpenLPT python=3.10
conda activate OpenLPT
pip install -r requirements.txt
```

Run a tracking job from the command line once a `config.txt` exists:

```
openlpt path/to/project/config.txt
```

The GUI's **Tracking** view does the same under the hood via `QProcess.start("python -m openlpt config.txt")` and streams stdout to a live log panel.

Adapting OpenLPT to your data

CONFIGURATION ANATOMY

`config.txt` is the master file: frame range, FPS, camera count, **view volume** (`xmin..zmax`), voxel-to-mm, output folder, object type (*Tracer* / *Bubble*), and pointers to `cam{i}.txt` + `cam{i}ImageNames.txt`. The chosen object type loads `tracerConfig.txt` or `bubbleConfig.txt` for STB parameters.

MAPPING YOUR EXPERIMENT

- **Camera files** - export from the Calibration view (plate, then wand + bundle adjust). For refractive setups, the *closest vs. farthest* coordinate convention matters - see `.agent/workflows/refractive-camera-calibration.md`.